CHAPTER-1 Data Handling using Pandas –I

Pandas:

- It is a package useful for data analysis and manipulation.
- Pandas provide an easy way to create, manipulate and wrangle the data.
- Pandas provide powerful and easy-to-use data structures, as well as the means to quickly perform operations on these structures.

Data scientists use Pandas for its following advantages:

- Easily handles missing data.
- It uses Series for one-dimensional data structure and DataFrame for multi-dimensional data structure.
- It provides an efficient way to slice the data.
- It provides a flexible way to merge, concatenate or reshape the data.

DATA STRUCTURE IN PANDAS

A data structure is a way to arrange the data in such a way that so it can be accessed quickly and we can perform various operation on this data like- retrieval, deletion, modification etc.

Pandas deals with 3 data structure-

1. Series
2. Data Frame
3. Panel

We are having only series and data frame in our syllabus.

Series

Series-Series is a one-dimensional array like structure with homogeneous data, which can be used to handle and manipulate data. What makes it special is its index attribute, which has incredible functionality and is heavily mutable.

It has two parts-

1. **Data part (An array of actual data)**
2. **Associated index with data (associated array of indexes or data labels)**

e.g.-

Index	Data
0	10
1	15
2	18
3	22

- ✓ We can say that **Series** is a labeled *one-dimensional array which can hold any type of data.*
- ✓ Data of **Series** is *always mutable*, means it can be changed.
- ✓ But the size of Data of **Series** is *always immutable*, means it cannot be changed.
- ✓ **Series** may be considered as a **Data Structure with two arrays** out which **one array** works as *Index (Labels)* and the **second array** works as *original Data.*
- ✓ ***Row Labels*** in Series are called *Index*.

Syntax to create a Series:

<Series Object>=pandas.Series (data, index=idx *(optional)*)

- ✓ Where data may be *python sequence (Lists)*, ndarray, scalar value or a python dictionary.

How to create Series with nd array

Program-

```
import pandas as pd
import numpy as np
arr=np.array([10,15,18,22])
s = pd.Series(arr)
print(s)
```

Here we create an array of 4 values.

Default Index

Output-

```
0  10
1  15
2  18
3  22
```

Data

How to create Series with Mutable index

Program-

```
import pandas as pd
import numpy as np
arr=np.array(['a','b','c','d'])
s=pd.Series(arr,
   index=['first','second','third','fourth'])
print(s)
```

Output-

```
first     a
second    b
third     c
fourth    d
```

Creating a series from Scalar value

To create a series from scalar value, an index must be provided. The scalar value will be repeated as per the length of index.

```
1  import pandas as pd
2  s = pd.Series(50, index =[0, 1, 2, 3, 4])
3  print(s)
4
```

```
0    50
1    50
2    50
3    50
4    50
dtype: int64
```

Creating a series from a Dictionary

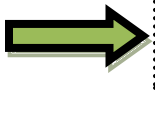
```
1  # import the pandas lib as pd
2  import pandas as pd
3
4  # create a dictionary
5  d = {'Name' : 'Hardik', 'Iplteam' : 'MI', 'Runs' : 1500}
6
7  # create a series
8  s = pd.Series(d)
9
10 print(s)
11
```

```
Name       Hardik
Iplteam        MI
Runs         1500
dtype: object
```

Mathematical Operations in Series

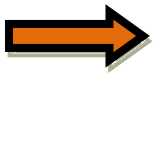
```
import pandas as pd
s=pd.Series([1,2,3,4,5])
print('To Multiply all values in a series by 2')
print('-----------------------------------------------------')
print(s*2)
print('To Find the Square of  all the values in a series ')
print('-----------------------------------------------------')
print(s**2)
print('To print all the values in a series that are greater than 2')
print('-----------------------------------------------------')
print(s[s>2])
```

```
To Multiply all values in a series by 2
-----------------------------------------------------
0     2
1     4
2     6
3     8
4    10
dtype: int64
```



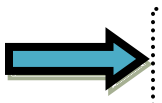
Print all the values of the Series by multiplying them by 2.

```
To Find the Square of  all the values in a series
-----------------------------------------------------
0     1
1     4
2     9
3    16
4    25
dtype: int64
```



Print Square of all the values of the series.

```
To print all the values in a series that are greater than 2
-----------------------------------------------------
2    3
3    4
4    5
dtype: int64
```



Print all the values of the Series that are greater than 2.

Example-2

```
import pandas as pd
s1=pd.Series([1,2,3,4,5],index=['a','b','c','d','e'])
s2=pd.Series([10,20,30,40,50],index=['a','b','c','d','e'])
s3=pd.Series([5,14,23,32],index=['a','b','c','d'])
print('To Add Series1 & series2')
print('-----------------------------------------------------')
print(s1+s2)
print('To Add Series2 & Series3')
print('-----------------------------------------------------')
print(s2+s3)
print('To Add Series2 & series3 and Filled Non Matching Index with 0')
print('-----------------------------------------------------')
print(s2.add(s3,fill_value=0))
```

```
To Add Series1 & series2
-----------------------------------------------------
a    11
b    22
c    33
d    44
e    55
dtype: int64
To Add Series2 & Series3
-----------------------------------------------------
a    15.0
b    34.0
c    53.0
d    72.0
e     NaN
dtype: float64
To Add Series2 & series3 and Filled Non Matching Index with 0
-----------------------------------------------------
a    15.0
b    34.0
c    53.0
d    72.0
e    50.0
dtype: float64
```

While adding two series, if Non-Matching Index is found in either of the Series, Then NaN will be printed corresponds to Non-Matching Index.

If Non-Matching Index is found in either of the series, then this Non-Matching Index corresponding value of that series will be filled as 0.

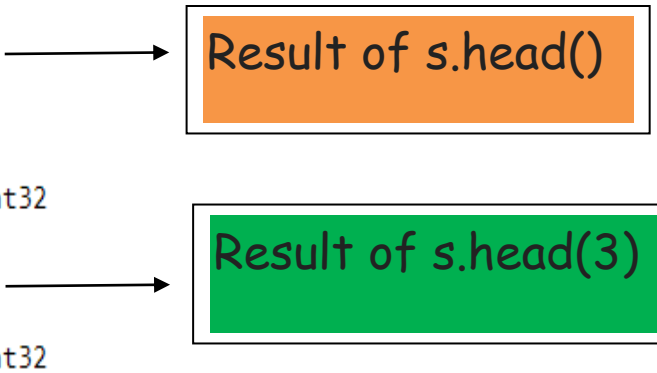
Head and Tail Functions in Series

head (): It is used to access the first 5 rows of a series.

Note :To access first 3 rows we can call series_name.head(3)

```
:
1  import pandas as pd
2  import numpy as np
3  arr=np.array([10,15,18,22,55,77,42,48,97])
4  # create a series from array
5  s = pd.Series(arr)
6  # to print fiest 5 rows
7  print (s.head())
8  # To print first 3 rows
9  print(s.head(3))
```

```
0    10
1    15
2    18
3    22
4    55
dtype: int32
0    10
1    15
2    18
dtype: int32
```



tail(): It is used to access the last 5 rows of a series.

Note :To access last 4 rows we can call series_name.tail (4)

```
import pandas as pd
import numpy as np
arr=np.array([10,15,18,22,55,77,42,48,97])
# create a series from array
s = pd.Series(arr)
# to print last 5 rows
print (s.tail())
# To print last 4 rows
print(s.tail(4))
```

```
4    55
5    77
6    42
7    48
8    97
dtype: int32
5    77
6    42
7    48
8    97
dtype: int32
```

Selection in Series

Series provides index label loc and ilocand [] to access rows and columns.

1. loc index label :-

Syntax:-series_name.loc[StartRange: StopRange]

Example-

```
1  import pandas as pd
2  import numpy as np
3  arr=np.array([10,15,18,22,55,77])
4  s = pd.Series(arr)
5  print(s)
6  print(s.loc[:2])
7  print(s.loc[3:4])
8  s.loc[2:3]
```

To Print Values from Index 0 to 2

To Print Values from Index 3 to 4

```
0    10
1    15
2    18
3    22
4    55
5    77
dtype: int32
0    10
1    15
2    18
dtype: int32
3    22
4    55
dtype: int32

2    18
3    22
dtype: int32
```

2. Selection Using iloc index label :-

Syntax:-series_name.iloc[StartRange : StopRange]

Example-

```
1  import pandas as pd
2  import numpy as np
3  arr=np.array([10,15,18,22,55,77])
4  s = pd.Series(arr)
5  print(s)
6  print(s.iloc[:2])
7  print(s.iloc[3:4])
8  s.iloc[2:3]
```

To Print Values from Index 0 to 1.

```
0    10
1    15
2    18
3    22
4    55
5    77
dtype: int32
0    10
1    15
dtype: int32
3    22
dtype: int32

2    18
dtype: int32
```

3. Selection Using [] :

Syntax:-series_name[StartRange> : StopRange] or

series_name[index]

Example-

```
1  import pandas as pd
2  import numpy as np
3  arr=np.array([10,15,18,22,55,77])
4  s = pd.Series(arr)
5  print(s)
6  print(s[1])
7  print('\n')
8  print(s[3:4])
9  s[:3]
```

To Print Values at Index 3.

```
0    10
1    15
2    18
3    22
4    55
5    77
dtype: int32
15


3    22
dtype: int32

0    10
1    15
2    18
dtype: int32
```

Indexing in Series

Pandas provide index attribute to get or set the index of entries or values in series.

Example-

```
:
1  import pandas as pd
2  import numpy as np
3  arr=np.array(['a','b','c','d'],)
4  s=pd.Series(arr,index=['first','second','third','fourth'])
5  print(s)
6  # To print only indexes in series
7  print('\n indexes in Series are:::')
8  print(s.index)
9
```

```
first     a
second    b
third     c
fourth    d
dtype: object

 indexes in Series are:::
Index(['first', 'second', 'third', 'fourth'], dtype='object')
```

Slicing in Series

Slicing is a way to retrieve subsets of data from a pandas object. A slice object syntax is –

SERIES_NAME [start:end: step]

The segments start representing the first item, end representing the last item, and step representing the increment between each item that you would like.

Example :-

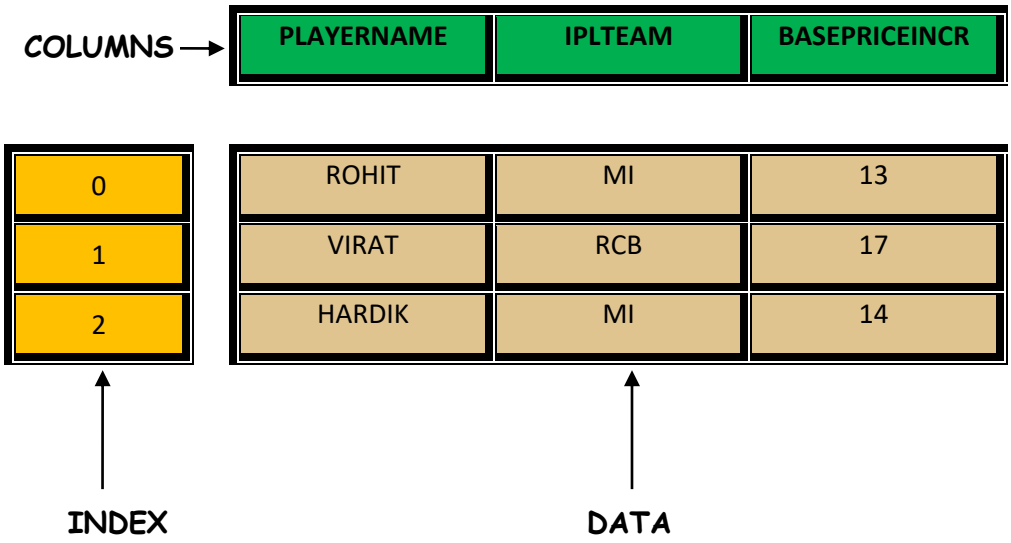
```
1  import pandas as pd
2  import numpy as np
3  arr=np.array([10,15,18,22,55,77])
4  s = pd.Series(arr,index=['A','B','C','D','E','F'])
5  print(s)
6  print(s[1:5:2])
7  print(s[0:6:2])
8
```

```
A    10
B    15
C    18
D    22
E    55
F    77
dtype: int32
B    15
D    22
dtype: int32
A    10
C    18
E    55
dtype: int32
```

DATAFRAME

DATAFRAME-It is a two-dimensional object that is useful in representing data in the form of rows and columns. It is similar to a spreadsheet or an SQL table. This is the most commonly used pandas object. Once we store the data into the Dataframe, we can perform various operations that are useful in analyzing and understanding the data.

DATAFRAME STRUCTURE



PROPERTIES OF DATAFRAME

1. A Dataframe has axes (indices)-
   - Row index (axis=0)
   - Column index (axes=1)
2. It is similar to a spreadsheet , whose row index is called index and column index is called column name.
3. A Dataframe contains Heterogeneous data.
4. A Dataframe Size is Mutable.
5. A Dataframe Data is Mutable.

A data frame can be created using any of the following-

1. **Series**
2. **Lists**
3. **Dictionary**
4. **A numpy 2D array**

How to create Empty Dataframe

```
: import pandas as pd
df=pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

How to create Dataframe From Series

Program-

```
import pandas as pd

s = pd.Series(['a','b','c','d'])

df=pd.DataFrame(s)

print(df)
```

Output-

```
   0
0  a
1  b
2  c
3  d
```

Default Column Name As 0

DataFrame from Dictionary of Series

Example-

```
import pandas as pd
name=pd.Series(['Hardik','Virat'])
team=pd.Series(['MI','RCB'])
dic={'Name':name,'Team':team}
df=pd.DataFrame(dic)
print(df)
```

```
     Name Team
0  Hardik   MI
1   Virat  RCB
```

DataFrame from List of Dictionaries

Example-

```
1  import pandas as pd
2  l = [{'Name': 'Sachin', 'SirName':'Bhardwaj'},
3       {'Name': 'Vinod', 'SirName':'Verma'},
4       {'Name': 'Rajesh', 'SirName':'Mishra'}]
5  df1=pd.DataFrame(l)
6  print(df1)
```

```
     Name   SirName
0  Sachin  Bhardwaj
1   Vinod     Verma
2  Rajesh    Mishra
```

Iteration on Rows and Columns

If we want to access record or data from a data frame row wise or column wise then iteration is used. Pandas provide 2 functions to perform iterations-

1. iterrows ()
2. iteritems ()

iterrows()

It is used to access the data row wise. Example-

```
1  import pandas as pd
2  l = [{'Name': 'Sachin', 'SirName':'Bhardwaj'},
3       {'Name': 'Vinod', 'SirName':'Verma'}]
4  df1=pd.DataFrame(l)
5  print(df1)
6  for(row_index,row_value) in df1.iterrows():
7      print('\n Row index is ::',row_index)
8      print('Row Value is::')
9      print(row_value)
```

```
     Name   SirName
0  Sachin  Bhardwaj
1   Vinod     Verma

 Row index is :: 0
Row Value is::
Name          Sachin
SirName     Bhardwaj
Name: 0, dtype: object

 Row index is :: 1
Row Value is::
Name         Vinod
SirName      Verma
Name: 1, dtype: object
```

iteritems()

It is used to access the data column wise.

Example-

```
1  import pandas as pd
2  l = [{'Name': 'Sachin', 'SirName':'Bhardwaj'},
3       {'Name': 'Vinod', 'SirName':'Verma'}]
4  df1=pd.DataFrame(l)
5  print(df1)
6  for(col_name,col_value) in df1.iteritems():
7      print('\n')
8      print('Column Name is ::',col_name)
9      print('Column Values are::')
10     print(col_value)
```

```
     Name   SirName
0  Sachin  Bhardwaj
1   Vinod     Verma


Column Name is :: Name
Column Values are::
0    Sachin
1     Vinod
Name: Name, dtype: object


Column Name is :: SirName
Column Values are::
0    Bhardwaj
1       Verma
Name: SirName, dtype: object
```

Select operation in data frame

To access the column data ,we can mention the column name as subscript.

e.g. - df[empid] This can also be done by using df.empid.

To access multiple columns we can write as df[[col1, col2,---]]

Example -

```
import pandas as pd
empdata={ 'empid':[101,102,103,104,105,106],
          'ename':['Sachin','Vinod','Lakhbir','Anil','Devinder','UmaSelvi'],
          'Doj':['12-01-2012','15-01-2012','05-09-2007','17-01- 2012','05-09-2007','16-01-2012'] }
df=pd.DataFrame(empdata)
print(df)
```

```
   empid     ename         Doj
0    101    Sachin   12-01-2012
1    102     Vinod   15-01-2012
2    103   Lakhbir   05-09-2007
3    104      Anil  17-01- 2012
4    105  Devinder   05-09-2007
5    106  UmaSelvi   16-01-2012
```

```
>>df.empid or df['empid']
0    101
1    102
2    103
3    104
4    105
5    106
Name: empid, dtype: int64

>>df[['empid','ename']]
   empid          ename
0    101         Sachin
1    102          Vinod
2    103        Lakhbir
3    104           Anil
4    105       Devinder
5    106       UmaSelvi
```